

GENTLENESS

March 13, 2026

LLM-Augmented Security Monitoring: Real-Time Log Analysis and Anomaly Detection

1 Introduction

It is widely recognized that some of the data collected, processed and stored in current information systems is of a personal nature, making it sensitive and requiring appropriate protection. Organizations, whether public or private, e.g., in domains like healthcare, finance, logistics that handle such data are consequently subject to increasingly stringent regulations. It is then crucial to equip information systems with mechanisms that control and restrict access to sensitive data to authorized users only. While experts in cybersecurity are involved in the design of advanced mechanisms to ensure data protection, an increasing number of individuals and organizations with limited technical expertise are tasked with managing large volumes of sensitive data. In most cases, these non-expert users are actually more aware of the operational realities of their domain and often have a more accurate and detailed vision of the field. However, the design and implementation of security policies in the actual information systems remain complex tasks that typically require significant expertise in formal security models. This complexity creates a gap between non-expert stakeholders, who are able to express their security requirements in natural language or through informal specifications, and the technical mechanisms needed to enforce these requirements effectively. As a result, there is a risk that the intended security objectives are either misinterpreted or only partially implemented, leading to vulnerabilities or inconsistencies.

Modern IT infrastructures generate **terabytes of logs daily**, but traditional security monitoring faces critical challenges:

- **Volume:** Human analysts cannot manually review millions of events
- **Complexity:** Attacks span multiple systems with subtle patterns
- **False Positives:** Rule-based systems (SIEMs) generate excessive noise
- **Evolving Threats:** New attack vectors emerge faster than signature updates

1.1 Motivation

LLMs offer **transformative potential** for security monitoring by:

- Understanding **natural language logs** across heterogeneous systems
- Detecting **semantic anomalies** (not just syntactic patterns)
- Generating **human-readable explanations** for alerts
- Adapting to **new attack patterns** without manual rule updates

1.2 Objectives

Develop an LLM-augmented log analysis system that:

1. Ingests and normalizes logs from multiple sources (firewalls, IDS, authentication systems)
2. Detects anomalies using **semantic analysis** (beyond traditional signature matching)
3. Generates **actionable insights** with natural language explanations
4. Provides **real-time threat scoring** based on context
5. Enables **interactive investigation** via LLM-powered query interface

2 System Design

2.1 Inputs

Table 1: System Input Data Sources

Source	Example Log Entry	Format
Firewall	Mar 13 14:23:45 firewall11 DROP TCP 192.168.1.100:54321 -> 10.0.0.5:22	Syslog
IDS	[**] [1:2001234:5] ET SCAN Potential VNC Scan [**] [Priority: 2] {TCP} 192.168.1.50:43210 -> 10.0.0.15:5900	CEF
Authentication	Mar 13 14:24:01 sshd[12345]: Failed password for root from 192.168.1.100 port 54321 ssh2	Syslog
Cloud API	{ "time": "2026-03-13T14:24:05Z", "event": "DeleteBucket", "user": "admin@company.com", "status": "success" }	JSON
Application	ERROR [Database] - Connection pool exhausted (active: 200, max: 200) at com.company.service.UserService.getUser(UserService.java:45)	Custom

Table 2: System Outputs

Type	Example	Format
Alert	{ "threat": "brute_force_attempt", "confidence": 0.95, "entities": ["192.168.1.100", "root"], "evidence": [...], "explanation": "Multiple SSH failed logins...", "recommended_actions": ["block_ip",...] }	JSON
Threat Score	{ "host": "10.0.0.5", "current_score": 87, "trend": "increasing", "top_risk_factors": ["unusual_login_times",...] }	JSON
Investigation Report	Natural Language Summary: Between 14:20-14:30, the system detected a coordinated attack originating from 192.168.1.100. The attack followed these stages: [1] Port scanning..., [2] SSH brute force..., [3] Successful RDP connection... Recommended: Isolate workstation- 05...	PDF/HTML

2.2 Processing Pipeline

1. Log Collection and Normalization

- Aggregate logs from multiple sources (Syslog, API endpoints, files)
- Parse and normalize into common schema
- Enrich with contextual data (asset inventory, user roles)

2. Semantic Analysis with LLM

- Convert log sequences into **embeddings** capturing semantic meaning
- Apply **temporal analysis** to detect unusual patterns across time
- Compare against **known attack patterns** (stored as embeddings)
- Generate **natural language explanations** for detected anomalies

3. Anomaly Detection

- **Statistical Methods:** Detect outliers in log frequencies/patterns
- **Semantic Similarity:** Compare current activity against baseline
- **Behavioral Analysis:** Model normal user/entity behavior
- **Threat Intelligence:** Correlate with known IoCs (Indicators of Compromise)

4. Threat Scoring

- Assign severity based on:
 - Anomaly confidence score (from LLM)
 - Affected assets' criticality
 - Attack progression stage
 - Historical context
- Generate prioritized alert queue

5. Investigation Support

- Natural language query interface (e.g., "Show all suspicious database accesses by contractors in the last 24 hours")
- Automated incident timeline generation
- Hypothesis testing (e.g., "Could this be related to the phishing campaign we saw yesterday?")

3 Technical Implementation

3.1 Core Components

Component	Implementation Details
Log Ingestion	Fluentd/Logstash with custom parsers for each log source
Normalization	Python-based schema enforcement and enrichment
Embedding Generation	Mistral-7B or sentence-transformers (all-MiniLM-L6-v2)
Vector Database	Weaviate or Qdrant for similarity search
Anomaly Detection	Combination of isolation forests and semantic similarity
Threat Intelligence	MISP feed integration for IoCs
User Interface	Streamlit dashboard with LLM chat interface

3.2 Novel Techniques

- **Log2Vec:** Custom embedding technique for log sequences that:
 - Preserves temporal relationships between events
 - Encodes structural patterns (e.g., failed login → successful login)
 - Handles multi-source correlations (e.g., firewall + auth logs)
- **Context-Aware Anomaly Scoring:**
 - Dynamic thresholds based on:
 - * Time of day (business hours vs. off-hours)
 - * User role and typical behavior patterns
 - * Asset criticality and recent changes
 - Uses LLM to generate context-specific explanations
- **Semantic Attack Pattern Matching:**

- Represents known attacks as embedding patterns
- Detects variations of known attacks (e.g., "living-off-the-land" techniques)
- Adapts to new attack vectors through continuous learning
- **Interactive Investigation:**
 - Natural language query interface for security analysts
 - Automated hypothesis generation and testing
 - "What-if" scenario simulation

3.3 Evaluation Metrics

The system will be evaluated using:

Metric	Description
Detection Accuracy	True/False Positive/Negative rates compared to labeled datasets
Mean Time to Detect (MTTD)	Average time between attack start and detection
Mean Time to Respond (MTTR)	Time from detection to analyst action (with vs. without LLM)
Alert Volume Reduction	Percentage reduction in false positives compared to traditional SIEM
Analyst Efficiency	Time saved on investigation tasks (measured via user studies)
Explainability Quality	Human evaluation of generated explanations (1-5 scale)

4 Datasets and Evaluation

4.1 Test Datasets

- **Synthetic Attacks:** Generated using:
 - MITRE ATT&CK frameworks for attack scenarios
 - GANs to create realistic but novel attack patterns
 - Red team exercises in controlled environments
- **Real-World Logs:**
 - Public datasets (e.g., Los Alamos National Lab, Honeynet Project)
 - Anonymized enterprise logs (if available)
 - CTF (Capture The Flag) competition datasets
- **Baseline Comparison:**
 - Traditional SIEM (Splunk, ELK)
 - Rule-based detection systems
 - Existing ML-based anomaly detection

4.2 Evaluation Scenarios

1. **Brute Force Attacks:** SSH/RDP password guessing
2. **Lateral Movement:** Attacker moving through internal network
3. **Data Exfiltration:** Unusual data transfers
4. **Privilege Escalation:** Unauthorized access to admin functions
5. **Insider Threats:** Malicious activity by authorized users
6. **Zero-Day Attacks:** Previously unseen attack patterns

5 Expected Contributions

This project will contribute:

- A **novel framework** for LLM-augmented security monitoring
- **New techniques** for semantic log analysis and anomaly detection

- **Empirical evaluation** of LLM performance in security contexts
- **Open-source tools** for log analysis and threat detection
- **Practical insights** for integrating LLMs into SOC operations

6 Challenges and Mitigations

Challenge	Description	Mitigation Strategy
Data Privacy	Logs may contain sensitive information	Anonymization, differential privacy, on-premise processing
Model Hallucinations	LLM may generate incorrect explanations	Human-in-the-loop validation, confidence scoring
Performance	Real-time processing of high-volume logs	Optimized embeddings, incremental analysis, sampling
Adversarial Attacks	Attackers may try to poison the model	Input validation, anomaly detection on model inputs
Explainability	Need for transparent decision-making	Attention visualization, rule extraction from LLM
Integration	Compatibility with existing SIEMs	Standard interfaces (Syslog, REST API), modular design

7 Future Work

Potential extensions include:

- **Automated Response:** LLM-guided playbook execution
- **Federated Learning:** Collaborative threat detection across organizations
- **Continuous Learning:** Online adaptation to new attack patterns
- **Multimodal Analysis:** Incorporating network traffic and endpoint data
- **Regulatory Compliance:** Automated audit report generation
- **Threat Hunting:** Proactive search for hidden threats using LLM